

TP VSION — Image Stitching

Points d'intérêt, descripteurs, appariement et homographie

Florian BARBE

Décembre 2025

1 Question 1 : Analyse du squelette de code

Le squelette fourni comporte :

- `MyFeatureDetector` : détection des points d'intérêt ;
- `MyDescriptorExtractor` : calcul des descripteurs ;
- `MyDescriptorMatcher` : appariement de descripteurs ;
- `main.cpp` : orchestration des étapes (lecture, détection, description, matching, homographie, stitching).

Le flux général est :

Image 1 / Image 2 → Détection → Description → Matching → Homographie → Stitching.

2 Question 2 : Création du détecteur ORB

Le détecteur ORB a été instancié via :

```
_myFeatureDetector = ORB::create();
```

ORB est choisi car :

- il est rapide ;
- il est robuste aux rotations ;
- il utilise un descripteur binaire compatible avec le matching Hamming.

```
// We detect keypoints in img1
_myFeatureDetector1.setImage(im1);
_myFeatureDetector1.detectFeatures();
imKP1 = _myFeatureDetector1.displayFeatures(Scalar(0, 255, 0));
```

Figure 1: Détection et affichage des keypoints

3 Question 3 : Affichage des points d'intérêt

Le code ajouté pour détecter et afficher les keypoints est :

Le premier paramètre du constructeur ORB correspond au nombre maximal de points détectés.

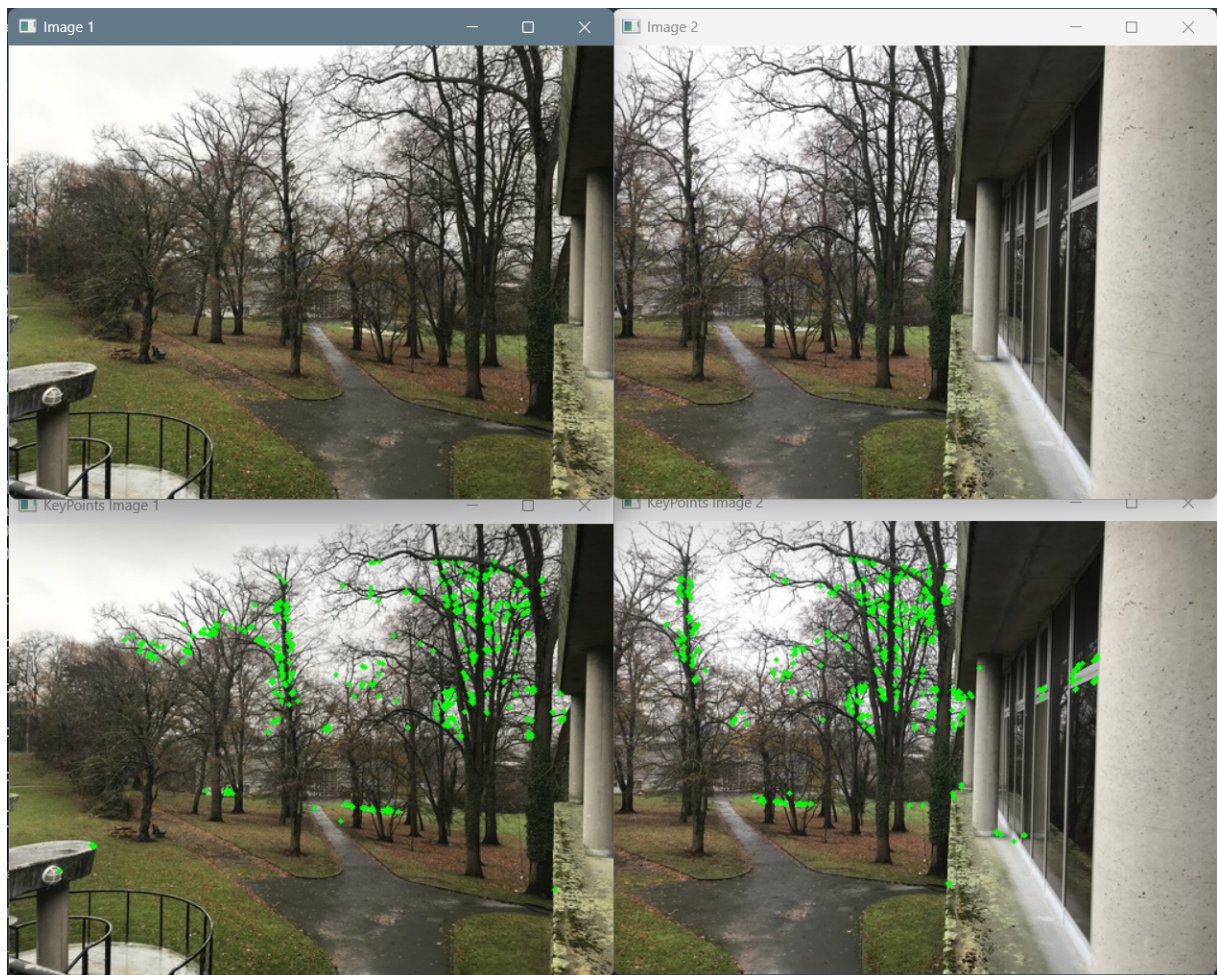


Figure 2: Points d'intérêt détectés pour un paramètre valant 500

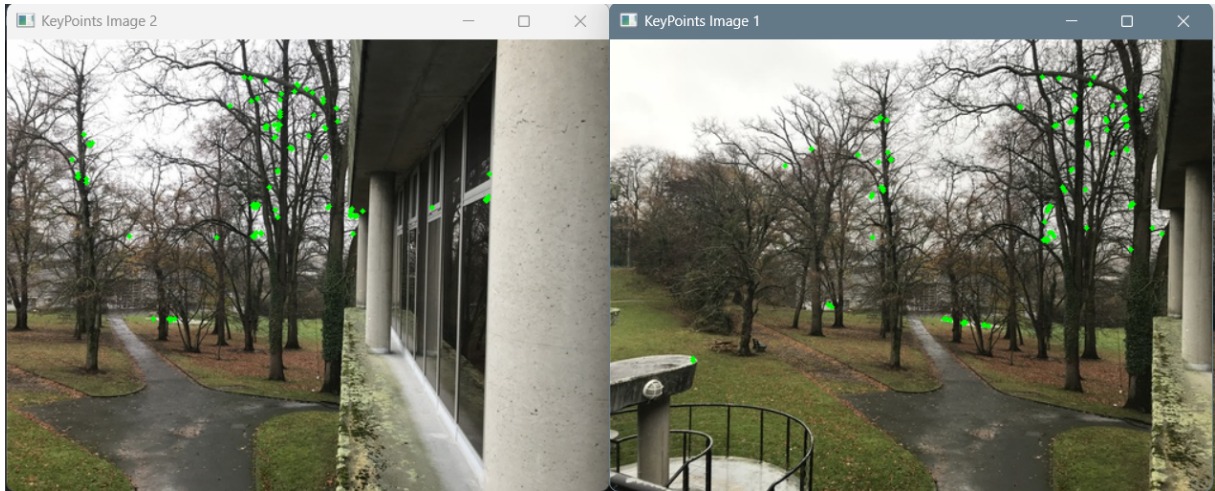


Figure 3: Avec un paramètre de 100

4 Question 4 : Instanciation des descripteurs

On utilise le descripteur ORB, qui est cohérent avec le détecteur ORB employé précédemment. ORB génère un vecteur binaire compact optimisé pour la vitesse et compatible avec le matching par distance de Hamming.

L'instanciation se fait en une ligne :

```
_myDescriptorExtractor = ORB::create();
```

5 Question 5 : Calcul des descripteurs

Dans `MyDescriptorExtractor.cpp` :

Une fois les points d'intérêt détectés, l'étape suivante consiste à calculer les descripteurs associés à chacun d'eux. Dans le fichier `MyDescriptorExtractor.cpp`, cette opération est réalisée grâce à la méthode :

```
_myDescriptorExtractor->compute(_myImage,
                                _myDetectedFeatures,
                                _myDescriptors);
```

6 Question 6 : Appariement

Une fois les descripteurs calculés pour les deux images, nous effectuons leur appariement à l'aide du *Brute Force Matcher* d'OpenCV. Celui-ci associe, pour chaque descripteur de l'image 1, le descripteur de l'image 2 ayant la plus faible distance.

J'ai défini un seuil pour filtrer les correspondances peu fiables. Conformément à la consigne du TP, nous conservons uniquement les matchs satisfaisant :

$$d_i \leq \alpha \cdot d_{\min},$$

où nous avons choisi ici $\alpha = 3$, donnant de bons résultats dans la plupart des configurations avec les images par défaut. Ce filtrage supprime la majorité des faux appariements tout en conservant des correspondances suffisamment robustes pour le calcul de l'homographie.

7 Question 7 : Affichage du matching

Le résultat final de l'appariement est obtenu via :

```
matchingResult = _myDescriptorMatcher.drawMatchingResults(  
    im1,  
    _myFeatureDetector1.getFeatures(),  
    im2,  
    _myFeatureDetector2.getFeatures(),  
    _myBestMatches  
);
```

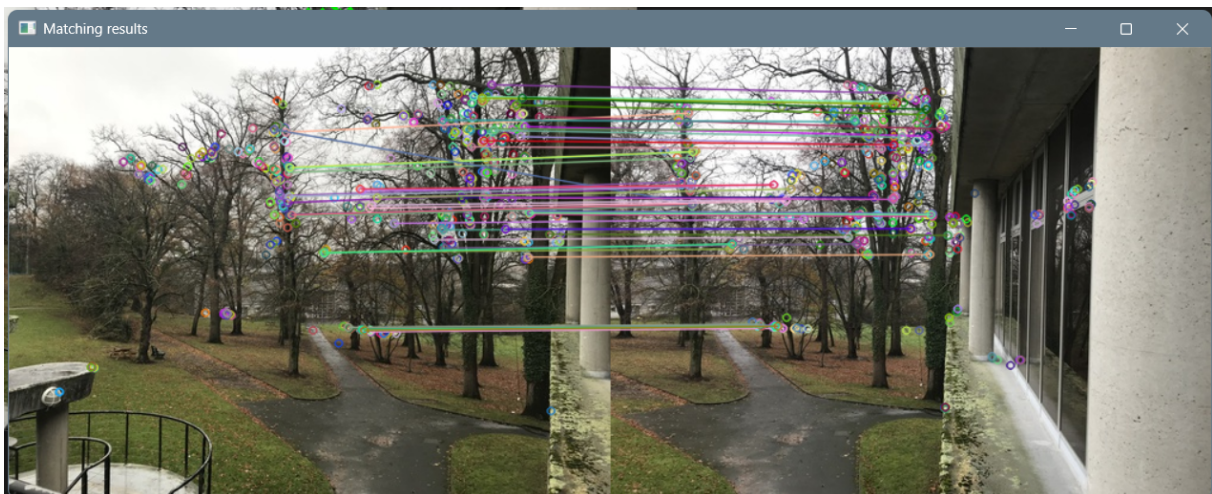


Figure 4: Résultats de la mise en correspondance.

Cet affichage permet de visualiser directement la qualité des correspondances retenues après le filtrage.

8 Question 8 : Extraction des points pour l'homographie

Les points correspondants sont extraits à partir des bons matches obtenus lors de l'appariement. Pour chaque correspondance, `queryIdx` désigne l'indice du keypoint dans l'image 1 et `trainIdx` l'indice du keypoint associé dans l'image 2 :

```
im1Pts.push_back(kpIm1[bestMatches[i].queryIdx].pt);  
im2Pts.push_back(kpIm2[bestMatches[i].trainIdx].pt);
```


Ces deux vecteurs contiennent donc des couples de points homologues :

$$G_M = \{p_i^{(1)}\}, \quad D_M = \{p_i^{(2)}\}.$$

Ils sont ordonnés de manière cohérente : le point $p_i^{(1)}$ correspond toujours au point $p_i^{(2)}$ associé par le matching. Cette structure est indispensable pour le calcul correct de l'homographie, qui repose sur ces paires de points en correspondance.

9 Question 9 : Calcul de l'homographie

Le code présenté ci-dessous correspond à l'étape finale du pipeline : le calcul de l'homographie entre les deux images puis la génération du résultat de stitching. Voici l'explication détaillée du fonctionnement de ce bloc :

- **Vérification du nombre de points** : `if(im1Pts.size() > 0)` vérifie que suffisamment de correspondances ont été trouvées pour pouvoir calculer une homographie (au moins 4 points sont nécessaires).
- **Calcul de l'homographie** : `findHomography(im2Pts, im1Pts, RANSAC, 1.5)` estime la matrice projective H qui transforme les points de l'image 2 dans le référentiel de l'image 1. L'utilisation de **RANSAC** permet de rejeter les faux appariements (outliers). Le seuil 1.5 correspond à la tolérance maximale d'erreur.
- **Warping de l'image 2** : `warpPerspective(im2, result, homography, Size(...))` applique la matrice H à l'image 2 afin de la reprojeter dans le plan de l'image 1. Un canvas plus large est créé pour accueillir les deux images côte à côte.
- **Copie de l'image 1** : `im1.copyTo(half)` place l'image 1 sur la partie gauche du canvas. L'image 2 transformée est déjà inscrite dans le reste de **result**.
- **Affichage du résultat** : `imshow("Warping", result)` affiche l'image finale composée, c'est-à-dire le stitching obtenu à partir de l'homographie.

Ainsi, ce bloc de code réalise le calcul de la transformation projective entre les deux images puis assemble automatiquement les deux vues dans un même repère géométrique.

```

// BELOW: FIND THE HOMOGRAPHY MATRIX AND STICH IMAGE
if(im1Pts.size()>0) {
    homography = findHomography( im2Pts, im1Pts, RANSAC, 1.5 );
    cout << "Homography: " << homography << endl;
    // Use the Homography Matrix to warp the images
    cv::Mat result;

    if(!homography.empty()){
        warpPerspective(im2,result,homography,cv::Size(im1.cols+im2.cols,im2.rows));
        cv::Mat half(result,cv::Rect(0,0,im1.cols,im1.rows));
        im1.copyTo(half);

        imshow( "Warping", result );
    }
}
// ABOVE: FIND THE HOMOGRAPHY MATRIX AND STICH IMAGE

```

Figure 5: Partie de code à commenter

10 Question 10 : Résultat du stitching



Figure 6: Résultat final du stitching.

L'homographie estimée permet d'aligner correctement les deux images dans leur zone de recouvrement, ce qui produit un panorama cohérent d'un point de vue géométrique. Cependant, plusieurs défauts apparaissent car la méthode utilisée repose uniquement sur l'homographie et ne comporte ni correction photométrique, ni équilibrage de couleur, ni blending multi-bandes.

11 Question 11 : Tests avec d'autres images

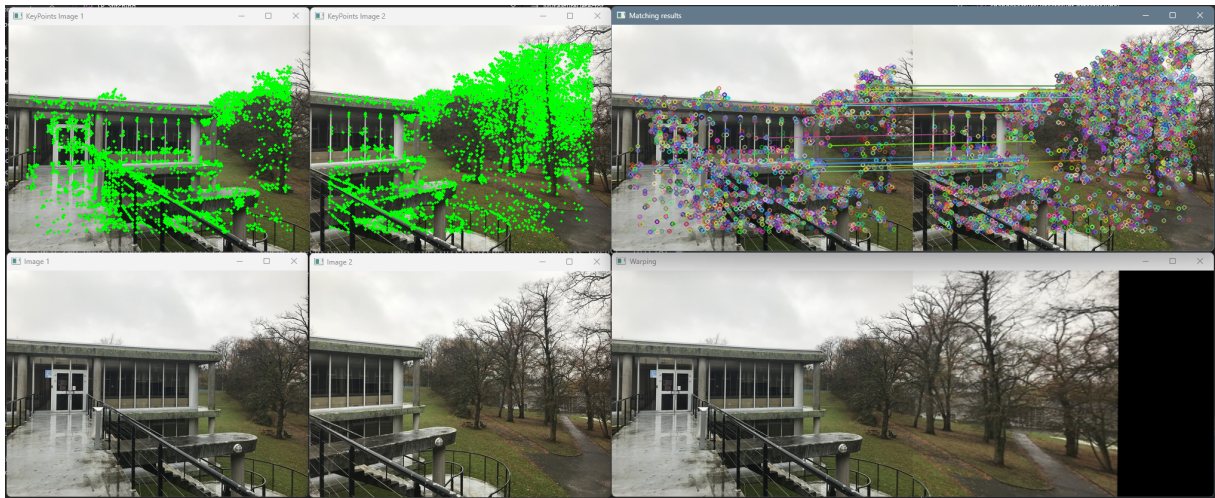


Figure 7: Avec une autre image



Figure 8: Avec une autre image (zoom sur le stitching)

Avec cette seconde paire d'images, RANSAC parvient à conserver suffisamment de correspondances fiables pour obtenir une homographie correcte : les éléments rigides (bâtiment, garde-corps) s'alignent globalement bien après warping.

Le flou visible sur la partie droite du warping provient du fait que `warpPerspective` rééchantillonne l'image selon une transformation projective. Dans les zones où la scène présente une forte profondeur (arbre, arrière-plan), l'homographie n'est plus adaptée : elle suppose que la scène est plane. Ainsi, des parties de l'image sont étirées ou compressées de manière non uniforme, ce qui entraîne un floutage perceptible même si l'image d'origine était nette.

12 Question 12 : Autres détecteurs et descripteurs

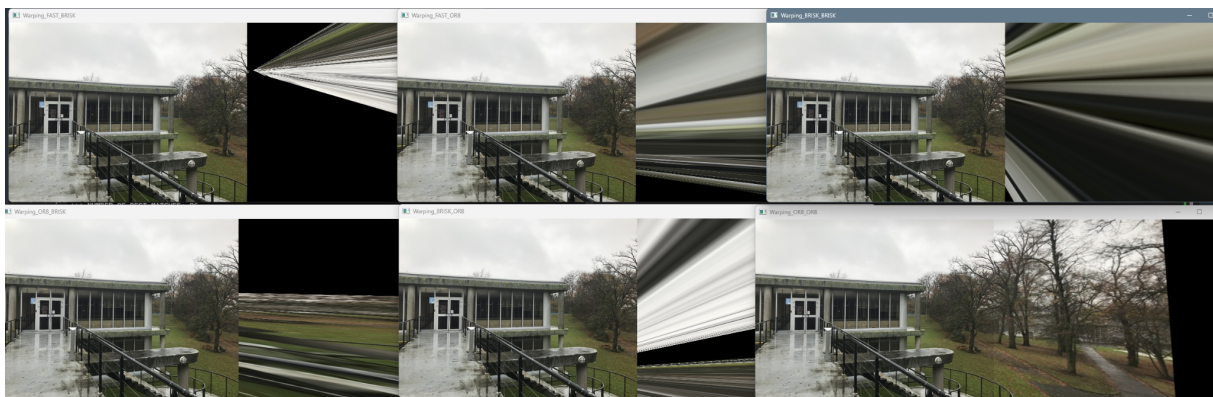


Figure 9: 6 tests, avec ORB ou BRISK pour détecteurs, et ORB, BRISK ou FAST pour extracteurs

Analyse des résultats de warping. La figure ci-dessus présente les résultats du warping obtenus pour toutes les combinaisons détecteur-descripteur (FAST-BRISK, FAST-ORB, BRISK-BRISK, ORB-BRISK, BRISK-ORB et ORB-ORB). On observe que plusieurs combinaisons conduisent à un fort étirement ou à une distorsion radiale de l'image de droite. Ce phénomène indique clairement que l'homographie estimée est erronée : dans ces cas, les appariements contiennent trop de correspondances aberrantes pour que `findHomography` puisse produire une transformation cohérente. Les déformations extrêmes (effet de “rayons” ou de “fuite vers un point”) témoignent typiquement d'une homographie dégénérée liée à des points incompatibles ou à un manque de recouvrement fiable entre les images.

À l'inverse, le couple ORB-ORB donne un résultat nettement plus stable : le warping est visuellement plausible et l'alignement global respecte la structure de la scène. ORB fournit ici un ensemble de points plus répétables et descripteurs plus robustes, ce qui limite les faux appariements et permet une estimation correcte de la matrice d'homographie.

Ces observations montrent que la qualité du warping dépend fortement de la cohérence du couple détecteur-descripteur. Les combinaisons hétérogènes FAST/BRISK, BRISK/ORB ou FAST/ORB ne produisent pas un appariement suffisamment fiable pour garantir une géométrie correcte.